Pokopia Habitats

Datapack & Add-on Author's Guide

How to write habitat definitions, extend the habitats that ship with the mod, and register your own spawns with a datapack or a companion mod. Works with vanilla blocks and any modded block or item.

For Pokopia Habitats (Cobblemon add-on) · NeoForge

Contents

1. How a habitat is detected

A habitat is a small build the mod recognises by the blocks inside a box. When a build matches one of the loaded definitions, the mod registers a habitat there and starts spawning the Pokemon listed in that definition, up to three at a time.

The box is a square footprint centred on the checked block, five blocks wide on X and Z, and five blocks tall counting upward from that block. Both sizes are server settings (`scannerBoxWidth` and `scannerBoxHeight`, each adjustable from 1 to 33), so a pack should aim for builds that fit inside five by five by five and never assume more room than that.

Two things decide a match:

- **Counting.** Every non-air block in the box is counted. A requirement is met when the number of matching blocks is at least its `count`. Blocks do not need to touch each other or sit in any pattern; they only need to be inside the box.
- **Room to stand.** The box must contain at least one open spot (a passable block with another passable block above it). A build sealed inside solid stone is never registered.

A build can satisfy several definitions at once. The mod sorts every definition by how many blocks and items it asks for and prefers the most demanding one, so a detailed build wins over a plain one that happens to share some blocks.

Liquids count as blocks. Water and lava are ordinary entries (`minecraft:water`, `minecraft:lava`), and a waterlogged block counts both as itself and as water, so a waterlogged fence still satisfies a water requirement.

2. Where the files live and how they are named

Each habitat is one JSON file. The mod reads every file under this path in any loaded datapack or mod resources:

```
data/<namespace>/pokopia/habitats/<path>.json
```

The file's id is the namespace plus the path that follows `pokopia/habitats/`, with the `.json` removed. Sub-folders are allowed and become part of the id.

File	Resulting id
`data/pokopia/pokopia/habitats/barn_pasture.json`	`pokopia:barn_pasture`
`data/mypack/pokopia/habitats/koi_pond.json`	`mypack:koi_pond`
`data/mypack/pokopia/habitats/floors/stone.json`	`mypack:floors/stone`

Two rules follow from this:

- **Give your own habitats your own namespace.** A file under `data/mypack/...` gets ids that start with `mypack:` and can never clash with the built-in ones.
- **To touch a built-in habitat, match its id.** The habitats shipped with the mod live in the `pokopia` namespace, so a file that adds to or replaces one must sit at `data/pokopia/pokopia/habitats/<same name>.json`. Sections 7 and 8 cover what happens then.

Names must use the resource-location character set: lowercase letters, digits, and the symbols `_ - . /`. No spaces and no capitals in the file or folder names.

3. The fields of a habitat file

A complete file looks like this. Only the parts your habitat needs have to be present; `blocks` (or `held_items`) and `spawns` are the ones that make a habitat useful.

```
{
  "name": "Koi Pond",
  "default_level": [5, 20],
  "dimensions": ["minecraft:overworld"],
  "biomes": ["#minecraft:is_river", "minecraft:swamp"],
  "blocks": [
    { "tag": "minecraft:planks", "count": 4 },
    { "key": "water", "count": 6, "any": ["minecraft:water"] }
  ],
  "held_items": [
    { "item": "minecraft:lily_pad", "count": 1 }
  ],
  "spawns": [
    { "pokemon": "magikarp", "rarity": "common", "chance": 0.6 },
    { "pokemon": "goldeen",  "rarity": "uncommon", "chance": 0.4 }
  ]
}
```

Field	Type	Meaning
`name`	text	Display name shown in the management menu. If left out, the mod builds one from the id.
`default_level`	number or [min, max]	Level, or level range, for any spawn that does not set its own. A single number means a fixed level.
`dimensions`	list of ids	Dimensions the habitat may form in. Leave it out for any dimension. See section 6.
`biomes`	list of ids or #tags	Biomes the habitat may form in. Leave it out to work everywhere. Accepts biome ids and biome tags.
`blocks`	list	Block requirements. See section 4.
`held_items`	list	Items that must be shown on item frames or armor stands. See section 5.
`spawns`	list	The Pokemon this habitat produces. See section 6.
`replace`	true / false	Discards anything gathered for this id before this file. See section 8.

A habitat with no valid block or item requirement is skipped. A habitat with no spawns still forms but stays empty.

4. Block requirements

Every entry in `blocks` is one requirement with a `count` (default 1). There are three ways to write one.

Single block

Match one specific block by its registry id.

```
{ "block": "minecraft:hay_block", "count": 5 }
```

A tag

Match any block in a block tag. Write the tag id under `tag`; the leading # is optional in this form. Tags are the tidy way to accept a whole family at once.

```
{ "tag": "minecraft:logs", "count": 4 }
```

A group of alternatives

Give the requirement a `key` and an `any` list. Any block or tag in the list counts toward the same requirement, so this reads as “any one of these”. Inside `any`, write tags with a leading #.

```
{
  "key": "seat",
  "count": 2,
  "any": [
    "minecraft:oak_stairs",
    "#minecraft:wooden_slabs",
    "farmersdelight:oak_cabinet"
  ]
}
```

The `key` is a label for the group. It matters when another pack wants to add more options to the same requirement later (section 7). For single `block` or `tag` entries the key defaults to the spec itself.

Counting notes

- The count is a total across the whole box, not per spec. A group asking for `count 2` is satisfied by two of any mix of its listed blocks.
- Air can be required on purpose with `minecraft:air`, `minecraft:cave_air`, or `minecraft:void_air`, which is useful for open cave-style habitats.
- An unknown block id is dropped with a warning. In a group, the known options still work, so listing a modded block next to a vanilla fallback is safe even where the mod is absent.

5. Held-item requirements

A habitat can ask for items on display, not just blocks. Entries in `held_items` are checked against items placed in item frames and items worn or held by armor stands inside the box. This is a way to key a habitat off something that has no block form, like a specific tool or trophy.

The three shapes match the block ones, using `item` instead of `block`:

```
"held_items": [
  { "item": "minecraft:fishing_rod", "count": 1 },
  { "tag": "minecraft:fishes", "count": 2 },
  { "key": "trophy", "count": 1, "any": [
      "minecraft:diamond", "minecraft:emerald", "somemod:gold_trophy"
  ] }
]
```

- Item counts add up stack sizes. A single frame holding a stack of items counts as that many toward the requirement.
- Only item frames and armor stands are read. Items in chests or barrels do not count.
- A habitat may use blocks, items, or both. The build must satisfy every requirement of both lists.

6. Spawn entries and rarity

Each entry in `spawns` describes one Pokemon the habitat can produce. Only `pokemon` is required.

Field	Type	Meaning
`pokemon`	text	A Cobblemon properties string. A plain species name works ("eevee"), and extra properties are allowed ("eevee shiny=yes", "gible level=10"). Any species Cobblemon knows, including add-ons, is valid.
`rarity`	text	Selection weight tier. One of common, uncommon, rare, ultra-rare (or epic), legendary. Defaults to common.
`weight`	number	Overrides the rarity tier's weight. Use only when you want a value between the tiers; leave it out otherwise.
`chance`	0 to 1	Once this entry is picked, the odds the spawn actually happens. Defaults to 1.
`level`	number or [min, max]	Level for this entry, overriding the habitat's default_level. A level written into the properties string wins over this.
`biomes`	list of ids or #tags	Restrict this one Pokemon to certain biomes, even if the habitat itself is open.
`dimensions`	list of ids	Restrict this one Pokemon to certain dimensions.

How a spawn is chosen

When a habitat has a free slot it rolls one entry, weighted by rarity, and then tests that entry's `chance`. Rarer tiers carry less weight, so they surface less often. The built-in weights are:

Rarity	Weight	Notes
common	100	The everyday residents of a habitat.
uncommon	45	Also the value used when no rarity is given a recognised name.
rare	15	Evolved or notable species.
ultra-rare / epic	5	Both names map to the same weight. Pseudo-legendaries, starters.
legendary	1	Reserved for the rarest sightings.

A spawn's level is decided in this order: a level inside the properties string, then the entry's `level`, then the habitat's `default_level`, then Cobblemon's own default for that species.

Dimension and biome gates

At the habitat level, `dimensions` and `biomes` decide where the build can form at all. Left out, they place no restriction and the habitat works anywhere. A per-spawn `dimensions` or `biomes` narrows a single Pokemon further, so one habitat can hand out different residents in different places. Dimension specs are plain ids (`minecraft:the_nether`). Biome specs are ids (`minecraft:plains`) or tags (`#minecraft:is_forest`).

7. Adding to a habitat that already exists

The mod does not keep only the top file for an id. It reads every file that shares an id, from the lowest-priority pack to the highest, and merges them. That is how a second pack extends a habitat without erasing the first. Nothing in the original file is lost unless you ask for it.

To add to a built-in habitat, place a file at its id and include only the parts you want to contribute. To append to the barn that ships with the mod, an add-on ships this at `data/pokopia/pokopia/habitats/barn_pasture.json`:

```
{
  "blocks": [
    { "key": "pen", "any": ["cozyfarm:birch_gate"] }
  ],
  "spawns": [
    { "pokemon": "skiddo", "rarity": "uncommon", "chance": 0.4 }
  ]
}
```

What this does when it merges on top of the original:

- The `skiddo` entry is added to the barn's spawn list. Existing spawns stay.
- Because the block entry reuses the key `pen`, its `birch_gate` is folded into the barn's existing “pen” group as another accepted option. A build can now satisfy the pen with the modded gate.
- No `count` is given, so the pen's required count is unchanged. Include `count` only when you mean to change it.

Merge rules in short:

- Requirements merge by `key`. Reusing a key adds options to that group; a new key adds a new requirement.
- `count` changes only when your file states it.
- Spawns are appended.
- `name`, `default_level`, `dimensions`, and `biomes` from a later file take over if present; otherwise the earlier values remain.
- To rewrite one group's options from scratch while leaving the rest of the habitat alone, add `"overwrite": true` to that group. It clears that group's current options before adding yours.

```
{
  "blocks": [
    { "key": "pen", "count": 4, "overwrite": true, "any": [
        "cozyfarm:birch_gate", "minecraft:oak_fence"
    ] }
  ]
}
```

8. Replacing a habitat completely

When you want none of the original to survive, add `"replace": true` at the top of your file. It discards everything collected for that id up to that point, so what you write becomes the whole definition. Put the replacing file in a pack that loads at or above the mod's own, which for a normal datapack or add-on it does.

```
{
  "replace": true,
  "name": "Barn Pasture",
  "default_level": [5, 16],
  "blocks": [
    { "block": "minecraft:hay_block", "count": 5 },
    { "key": "pen", "count": 3, "any": [
        "minecraft:oak_fence", "minecraft:oak_planks"
    ] }
  ],
  "spawns": [
    { "pokemon": "miltank", "rarity": "common", "chance": 0.6 },
    { "pokemon": "tauros",  "rarity": "uncommon", "chance": 0.4 }
  ]
}
```

- Use `replace` to retune a built-in habitat's spawn list or requirements wholesale.
- Use `replace` to switch a habitat off in practice: keep a requirement so it still loads, and give it an empty `spawns` list so it never produces anything.
- The difference from section 7 is scope. `overwrite` clears one group; `replace` clears the entire habitat.

9. Using modded blocks and items

Any block or item is referenced the same way, by its registry id `namespace:path`. There is no separate list of “allowed” blocks; if the game has it registered, a requirement can ask for it. That covers every modded block and item as well as vanilla.

```
"blocks": [
  { "block": "create:andesite_casing", "count": 3 },
  { "tag": "forge:storage_blocks/copper", "count": 1 },
  { "key": "seat", "count": 1, "any": [
      "minecraft:oak_stairs",
      "farmersdelight:oak_cabinet",
      "supplementaries:oak_bench"
  ] }
]
```

Two habits keep a cross-mod pack working on any load-out:

- **Pair modded ids with a vanilla fallback in a group.** If a listed id is not present, it is skipped and the other options in the group still work, so the habitat degrades quietly instead of breaking.
- **Prefer tags where a shared one exists.** Many mods contribute to common tags, so a single tag entry can accept content from several mods without naming each block.

A requirement whose every option is unknown is dropped with a warning, and a habitat left with no valid requirement at all is skipped. This never crashes the load; it only removes the part that could not resolve.

10. Worked examples

A. A new habitat in your own namespace

File: `data/mypack/pokopia/habitats/aviary.json`. Forms anywhere, in any dimension, and leans on tags so it accepts modded planks and leaves.

```
{
  "name": "Aviary",
  "default_level": [6, 22],
  "blocks": [
    { "tag": "minecraft:planks", "count": 4 },
    { "tag": "minecraft:leaves", "count": 6 },
    { "key": "perch", "count": 2, "any": [
        "#minecraft:wooden_fences", "minecraft:chain"
    ] }
  ],
  "spawns": [
    { "pokemon": "pidgey",   "rarity": "common",   "chance": 0.7 },
    { "pokemon": "starly",   "rarity": "common",   "chance": 0.6 },
    { "pokemon": "rufflet",  "rarity": "uncommon", "chance": 0.4 },
    { "pokemon": "rowlet",   "rarity": "rare",     "chance": 0.3,
      "level": [8, 20] }
  ]
}
```

B. A compatibility add-on for a built-in habitat

File: `data/pokopia/pokopia/habitats/coral_reef.json` in a mod that ships new coral blocks. It teaches the reef to accept those blocks and adds one resident, without disturbing the original.

```
{
  "blocks": [
    { "key": "coral", "any": [
        "deepblue:brain_coral_block",
        "deepblue:fan_coral_block"
    ] }
  ],
  "spawns": [
    { "pokemon": "mareanie", "rarity": "uncommon", "chance": 0.4,
      "biomes": ["#minecraft:is_ocean"] }
  ]
}
```

C. A full override

File: `data/pokopia/pokopia/habitats/graveyard.json`. Takes the built-in graveyard over completely with a new roster.

```
{
  "replace": true,
  "name": "Graveyard",
  "default_level": [6, 40],
  "blocks": [
    { "key": "grave", "count": 4, "any": [
        "minecraft:stone_bricks", "minecraft:cobblestone_wall",
        "minecraft:mossy_cobblestone"
    ] },
    { "key": "spooky", "count": 2, "any": [
        "minecraft:soul_torch", "minecraft:cobweb", "minecraft:wither_rose"
    ] }
  ],
  "spawns": [
    { "pokemon": "gastly",   "rarity": "common", "chance": 0.7 },
    { "pokemon": "duskull",  "rarity": "common", "chance": 0.6 },
    { "pokemon": "gengar",   "rarity": "rare",   "chance": 0.25 }
  ]
}
```

11. Field reference

Top level

Key	Type	Default
`name`	text	built from the id
`default_level`	number or [min, max]	Cobblemon default
`dimensions`	list of dimension ids	any dimension
`biomes`	list of biome ids or #tags	any biome
`blocks`	list of block requirements	none
`held_items`	list of item requirements	none
`spawns`	list of spawn entries	none
`replace`	true / false	false

Block or item requirement

Key	Type	Meaning
`block`	id	One block id (blocks list only).
`item`	id	One item id (held_items list only).
`tag`	id	A block or item tag. Leading # optional here.
`any`	list of ids / #tags	Group of alternatives; any one counts.
`key`	text	Group label used when packs merge. Defaults to the single spec.
`count`	number	How many matching blocks or items are needed. Default 1.
`overwrite`	true / false	Clears this group's current options before adding. Default false.

Spawn entry

Key	Type	Meaning
`pokemon`	text	Cobblemon properties string. Required.
`rarity`	text	common, uncommon, rare, ultra-rare / epic, legendary. Default common.
`weight`	number	Overrides the rarity weight. Ignored when 0 or absent.
`chance`	0 to 1	Odds the spawn happens once picked. Default 1.
`level`	number or [min, max]	Level for this entry. Overrides default_level.
`biomes`	list of ids / #tags	Limit this Pokemon to these biomes.
`dimensions`	list of ids	Limit this Pokemon to these dimensions.

Box and pacing defaults

These come from the server config and can be changed by the pack's players, so treat them as the out-of-the-box values to design against.

Setting	Default	Meaning
`scannerBoxWidth`	5	Footprint width on X and Z (1 to 33).
`scannerBoxHeight`	5	Box height, counted upward (1 to 33).
`maxPokemonPerHabitat`	3	Living residents a habitat holds at once (1 to 3).
`habitatSpawnIntervalSeconds`	20	Seconds between spawn rolls for a free slot.

12. What the log tells you

When resources load, the mod reports what it read and why anything was dropped. Check the server log after a world load or a `/reload` to confirm a pack took effect.

- **Loaded N Pokopia habitat definitions.** The final count of habitats after merging. If your file is present and valid, the number rises.
- **requirement '<key>' has no valid blocks, skipped.** Every option in that group failed to resolve. Usually a typo in an id or a mod that is not installed.
- **unknown block '<id>' / bad tag id.** One spec did not resolve. Harmless inside a group with other working options; fatal to the group only if it was the sole option.
- **has no spawns; it can still be detected but stays empty.** The habitat loaded but its spawn list is empty. Expected when you switch a habitat off on purpose.

A working loop while authoring: edit the file, run `/reload`, read the log line, then place or rescan the build in the world to see the habitat register.